

DiffCI Open Evidence Study 2026

What change-aware test selection is actually worth on real open-source CI, measured, including where it is worth nothing

Published by DiffCI. Evidence window 2026-08-19 to 2026-09-03. Version v1, as of 2026-09-03.

Every repository named in this study is a public open-source project with public CI, analysed from its public source and replayed in DiffCI's own sandbox. None of them is a DiffCI customer, partner or user. None has installed DiffCI, been contacted about it, or endorsed anything here. No change was ever proposed to, or made in, any of their real CI pipelines.

Between 19 August and 3 September 2026, DiffCI ran a sequence of pre-registered experiments to answer one question: on real repositories, how much CI work can a dependency-graph test selector safely skip, and is that worth more than a trivial path rule? This study publishes every number those experiments produced, including the ones that argue against the product.

Headline findings

2,000

real commit deltas across 20 repositories, historically replayed

29.8%

of deltas offered any chance to be selective at all

94.3%

of those chances DiffCI selected fewer tests than a path rule; the path rule won 0

4.2%

aggregate test reduction against the path rule across the whole corpus

96.4% vs 70.7%

job-level recall of 280 real historical CI failures, DiffCI vs path rule

44% and 90%

job-level reduction on the two repositories where selections were actually executed and timed

0 of 32

false greens across every measurable mutation, in five corpora

0

production CI runs ever skipped, cancelled or blocked

The advantage is real, narrow, and concentrated in small repositories

The largest experiment replayed 2,000 real commit deltas, 100 each from 20 repositories chosen for structural diversity before any were run, through a real Cloudflare orchestrator. Each delta was classified, independently of DiffCI's own output, by whether a selective decision was even possible: 60.3% touched a lockfile, workflow or root configuration and

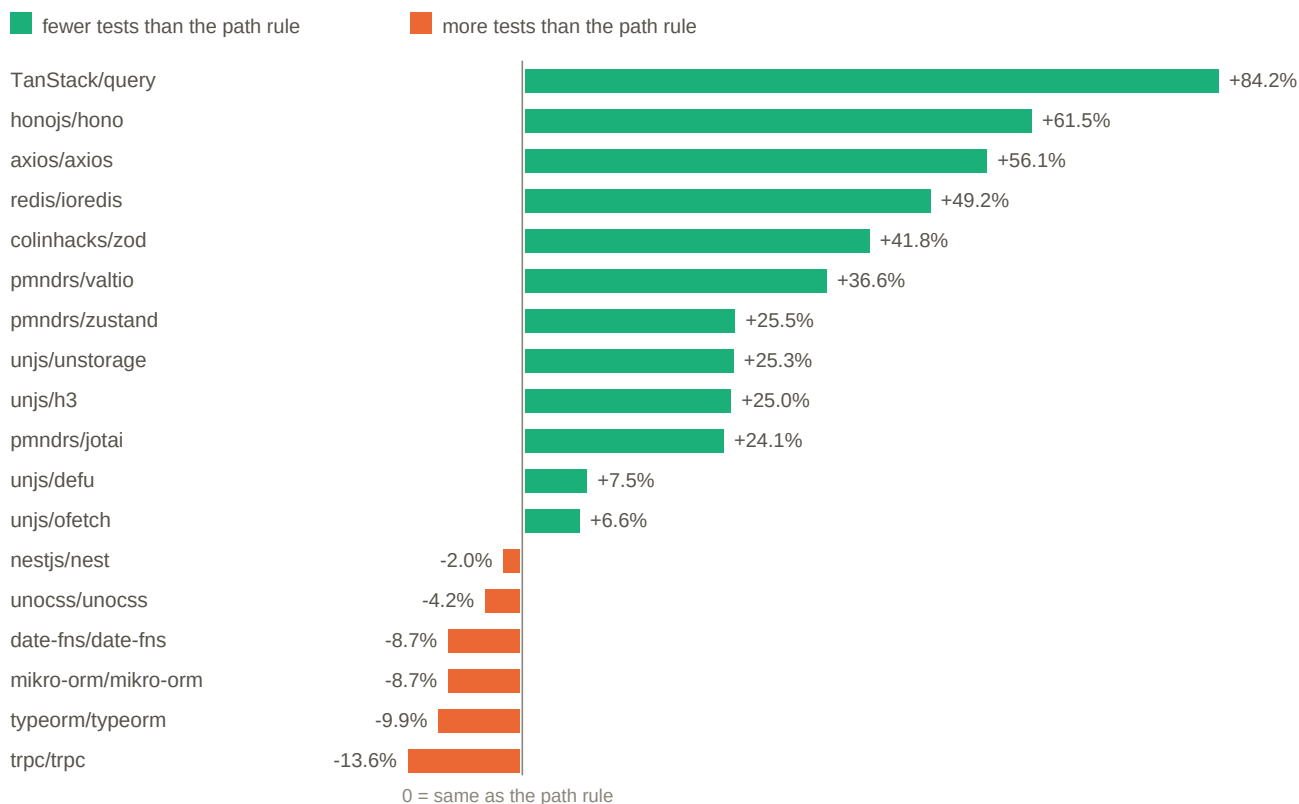
fell back to full CI by rule; 9.9% were already optimally handled by a plain path rule; 29.8% offered a genuine discriminative opportunity.

Within those 565 opportunities, DiffCI selected fewer tests than the path rule on 94.3%, tied on the rest, and lost on none. The median reduction against the path rule inside that bucket was 95.0%.

Across the whole corpus, weighted by test count, that collapses to a 4.2% aggregate reduction against the path rule. The unconditional median task reduction was 0.0%. Seven of the twenty repositories hit 100% mandatory fallback or unsafe graph confidence and contributed nothing but drag. A repository-clustered bootstrap puts the 95% interval on the aggregate at -4.8% to +47.9%, which crosses zero: this sample cannot rule out a null population effect.

The clearest correlate was repository size. The ten repositories below the median source-file count showed a 44.9% aggregate reduction; the nine above it showed -0.95%, indistinguishable from zero. Size and fallback rate are confounded in a sample this small, and the report says so.

Aggregate test reduction against the path rule, per repository



Stage 0, 100 deltas per repository, test-count weighted. Negative values are repositories where 100%-fallback deltas selected the full suite while the path rule had already narrowed. execa and ky are omitted: execa's path rule selected nothing, and ky's AVA test counts are unreliable.

Repository	Fallback rate	Discriminative opportunities	Aggregate reduction vs path rule
TanStack/query	12%	64	84.2%
axios/axios	33%	44	56.1%
colinhacks/zod	6%	83	41.8%
date-fns/date-fns	100%	0	-8.7%
honojs/hono	19%	81	61.5%
mikro-orm/mikro-orm	100%	0	-8.7%

nestjs/nest	100%	0	-2.0%
pmndrs/jotai	41%	48	24.1%
pmndrs/valtio	45%	41	36.6%
pmndrs/zustand	32%	18	25.5%
redis/ioredis	47%	46	49.2%
sindresorhus/execa	100%	0	n/a
sindresorhus/ky	12%	0	n/a
trpc/trpc	100%	0	-13.6%
typeorm/typeorm	100%	0	-9.9%
unjs/defu	79%	14	7.5%
unjs/h3	13%	67	25.0%
unjs/ofetch	64%	27	6.6%
unjs/unstorage	55%	32	25.3%
unocss/unocss	100%	0	-4.2%

Stage 0 per-repository results. Fallback rate is the share of the 100 deltas that fell back to full CI by rule or by unsafe graph confidence.

Safety held at scale, and it was not perfect

Using authenticated GitHub Actions history, the same 2,000 deltas yielded 280 real job-level CI failures across 154 deltas. DiffCI's plan would have missed 10 of them; the path rule would have missed 82. That is a job-level failure recall of 96.4% against 70.7%. Ten misses is a real finding. Earlier partial snapshots had suggested zero, and the final report refused to round to it.

Where a regression could be manufactured and measured, the record is cleaner. Across five corpora, every mutation whose detection was measurable was detected: 32 of 32, with zero false greens. That sum spans different harness generations and is not a reliability estimate. Twenty-five of the cases come from the canonical Linux container on hono and zod, whose frozen bundle prints its own caveat that 25 cases cannot support a rate.

The one place DiffCI's prediction was honestly weak is its own repository. A pre-flight predictor replayed leakage-safe against 24 historical failures in DiffCI's own CI scored a prevention recall of 0.696, and missed all nine unit-test failures in the set. The first run had scored 0.958, was investigated because it was too good, and turned out to be counting a registry declaration as a prediction.

Corpus	Environment	Measurable cases detected	False greens
cal.com	DiffCI sandbox	3 of 3	0
deepseek-ai/deepseek-harness	DiffCI sandbox	3 of 3	0
honojs/hono	canonical Linux container	20 of 20	0
colinhacks/zod	canonical Linux container	5 of 5	0
jest-community/eslint-plugin-jest	canonical Linux container, sealed target	1 of 1	0

Measurable mutation cases by corpus. A case is measurable only if reverting the change fails the full suite; unmeasurable cases are counted and

When the selection is executed, install cost decides the number

A computed selection is a claim. On two repositories the claim was tested: the full test suite and DiffCI's selected subset were both actually run and timed at the same commit, ten merges in total, and a runtime invariant checked whether the test runner honoured the selection instead of quietly broadening it. It did on 12 of 13 executions; the thirteenth was withheld rather than counted.

On cal.com, five merges chosen by a rule written down before their costs were known produced a test-stage reduction of 86.0% to 91.6%. Measured against the whole job, one merge came to 44.2%, because cal.com spends 319.7 seconds on install and 17.1 on prisma generate before a single test runs, and those are paid identically on both paths. On deepseek-harness, whose install takes 23 to 39 seconds, the same selection quality produced 79.5% to 89.5% at job level.

That forty-point gap between the two repositories has nothing to do with how good the selection was. It is entirely the ratio of install to test. Anyone quoting a single industry percentage has not measured the repository in question.

Repository	Merges executed	Test-stage reduction, net	Job-level reduction, net	Selection honoured	Measurable regressions caught
cal.com	6	86.0% to 91.6%	44.2%	8 of 8	3 of 3
deepseek-ai/deepseek-harness	4 of 5	83.5% to 94.3%	79.5% to 89.5%	3 exact, 1 framework-expanded, 1 withheld	3 of 3

Both paths executed and timed in DiffCI's sandbox. Net figures subtract DiffCI's own analysis cost. deepseek-harness carries 16 to 18 pre-existing failures on every run; a naive check would have read that as a caught regression every time, and the harness was fixed before it could.

Against a cheap path rule, DiffCI sometimes loses

Beating the full suite is a soft target. The business question is whether DiffCI's analysis pays for itself against a path rule that costs nothing to compute. Every CPU-second of DiffCI's own analysis is charged to DiffCI; the comparator is charged nothing, even though it reads the same graph.

On honojs/hono, measured in the canonical container across 22 candidates, DiffCI's gross saving against the full suite was +1,408.92 CPU-s and its incremental result against the path rule was -80.90 CPU-s: 4 candidates positive, 18 negative. Wherever the path rule was tight, DiffCI lost; every one of the four wins was a case where the path rule blew up to 123 tests and DiffCI held at 83. DiffCI paid a fixed toll of roughly 3.3 CPU-s per candidate to insure against that, and on hono the insurance cost more than it saved. Test counts had predicted this, with 12 of 20 selections classed as over-broad, and the CPU meter agreed. A meter that reported DiffCI winning everywhere would have indicted itself.

On colinhacks/zod the sign flipped: across five confirmed cases DiffCI selected 629 tests where the path rule selected 761 and the full suite held 967, with four of five selections classed efficient. On jest-community/eslint-plugin-jest, three candidates drawn blind under pre-committed rules cost DiffCI 4 to 6 CPU-s where the comparator cost 52 to 77 and the full suite 59 to 85, an incremental result of +71.48, +43.81 and +53.56 CPU-s.

The one out-of-sample test of the economic predictor was immerjs/immer, selected by a third party before any DiffCI data existed. The rule, frozen beforehand, predicted a positive sign; the measurement was +46.97 CPU-s, positive. That is $n = 1$, and it validates the ordering of the experiment rather than a prediction accuracy rate. The magnitude was not validated: on a matched subset the model had predicted +85.19, and the gap exposed that the linear per-file cost

model under-prices 1- and 2-file selections by roughly four times, an error whose direction inflates predicted savings.

Repository	Candidates	Full suite	Path rule	DiffCI incl. analysis	Incremental vs path rule
honojs/hono	22	1,928.94	439.12	520.02	-80.90
immerjs/immer	7 measured	79.02	79.51	32.54	+46.97
eslint-plugin-jest, GENERATION_C_01	1				+71.48
eslint-plugin-jest, MI-01	1				+43.81
eslint-plugin-jest, MI-02	1	64.30	59.88	6.32	+53.56

All values in CPU-seconds. Incremental CPU is comparator cost minus DiffCI's selected-run cost minus DiffCI's analysis cost. Positive means DiffCI's intelligence paid for itself against a free path rule; negative means it did not. All in the canonical Linux container (cloudflare/sandbox:0.12.5, Node 22). Blank cells are per-arm figures the source summary does not restate; across the three eslint-plugin-jest candidates the full suite cost 59 to 85, the path rule 52 to 77 and DiffCI 4 to 6.

The dependency graph has reached tests the diff could not name, but it has not been shown to be necessary

On the one blind-drawn candidate where detection, graph contribution and economics were all measurable at once, DiffCI selected 2 files of 174 against the path rule's 164, caught the regression, and saved 71.48 CPU-s. Both selected files were files the commit had changed. Nothing was reached through the graph. A rule that runs only the files a commit touched would have produced the identical selection, detection and cost.

On two other candidates, drawn independently under a different rule, DiffCI did reach a test file the commit had not touched, through the import graph, selecting 2 of 159 against the path rule's 149. On both, detection was unmeasurable because reverting the change did not fail the full suite. So the evidence ladder closes with one measured safety-plus-savings result, two independent demonstrations of graph reach, and no case where both were measured together.

The consequence is written into the project's standard: beating the full suite no longer counts, and beating the path rule is necessary but not sufficient. Any claim that the graph earns its compute must beat a direct-only rule, the cheap heuristic that has tied twice.

Most repositories cannot yet be reached at all

The narrowest finding in the study is how few repositories DiffCI can currently reason about. Of npm's 40 most-depended-upon packages, 8 have no tsconfig.json and are refused before any graph is built. Of the 28 that could be measured, 20 have a mapping density under 20%: fewer than one test file in five can reach any production file through the static import graph. At least two of the four mechanisms behind that band are DiffCI's own limitations, unresolved workspace self-references and monorepo scope, and the report declines to blame the repositories for them.

Canonical qualification, which requires a repository's suite to go green twice in the reference container before any selection evidence counts, has been attempted on only 7 of 32 repositories in the frozen frame: 3 went green twice. Seventeen have never been attempted. The funnel's honest headline is how little is known, not a green rate.

The external-validation sequence made this concrete. Five repositories were selected by a third party before any DiffCI inspection: 1 of 5 passed the pre-registered assessability gates. Fastify and Axios had red baselines; date-fns could not have its monorepo scope preserved; Chalk uses AVA, which is unsupported. Only immer was assessable.

The product's CI-reproduction engine, which must reconstruct a repository's real pipeline before it may plan against it,

refused on first exposure to eslint/eslint because the repository commits no lockfile and runs a bare npm install. The refusal was correct on the facts: the reference arm ran 38,627 tests green, and the engine still declined to claim it could account for an unpinned install. The first mechanically selected pilot of four further repositories reproduced none: rimraf was rejected at screening, lodash, husky and chalk were refused for three distinct coverage gaps. A weaker, distinctly labelled time-boxed dependency basis built afterwards took eslint at its pinned commit to REPRODUCED, 38,647 tests in both arms, while still failing loudly on the live head that had drifted past the cutoff.

Gate	Outcome	Source
No tsconfig.json in the tree	8 of 40 refused before analysis	mapping-density survey
Static import graph connects tests to production	20 of 28 measured under 20% mapping density	mapping-density survey
Suite green twice in the canonical container	3 of 7 attempted; 17 of 32 never attempted	qualification funnel
Pre-registered external assessability gates	1 of 5 passed	external validation #1
CI pipeline reproduced from real configuration	0 of 4 in the first pilot; eslint REFUSED then REPRODUCED under a time-boxed basis	engine coverage baseline

Where repositories stop. Each row is a different gate; they are not a single funnel and the denominators differ.

Methodology and research integrity

Every experiment in this study was pre-registered: the methodology, corpus, thresholds or prediction was committed to the repository, and in the larger cases SHA-256 hashed, before the run that would be graded against it. Stage 0's methodology was frozen at one commit across 13 files; the 24-hour shadow-ramp gate was declared before the repositories it governed were enrolled; the immer economic prediction was committed before any economics arm ran; the GENERATION_C target was sealed before any DiffCI output existed.

Measurements that matter were made in one canonical environment, a Cloudflare sandbox container (cloudflare/sandbox:0.12.5, Ubuntu 22.04.5, Node 22.23.2), with the agent's exact tarball bytes verified in-container before anything was measured, and results frozen as checksummed bundles before interpretation. Developer-host evidence is labelled as such and was not used for CPU, wall time or economics.

Where DiffCI could not measure, it abstained: results are reported as UNMEASURABLE, REFUSED or withheld, never estimated into the denominator. Where the process went wrong, the correction is recorded next to the original rather than edited over it.

- One Stage 0 delta with an impossible count was excluded from aggregates rather than patched mid-benchmark on an unconfirmed root cause.
- The pre-flight replay's first result of 0.958 was rejected as too good, the counting defect was found, and 0.696 was reported.
- The qualification-funnel script initially reported 12 analyser-ineligible repositories; a classification defect was fixed and 8 reported.
- GENERATION_C_01's corpus row reports one graph-derived selection; the row was left unedited and re-read on the page as 0 graph-derived, 2 directly changed.
- A mocha output parser was found to have been validated against a hand-typed fixture instead of real ANSI-wrapped output; the regression test now uses the captured bytes.
- A cost figure that had been transcribed as \$0.0044 in one place and \$0.0045 in another was re-measured rather than reconciled by choosing one.

What this study does not establish

This section is part of the study, not a footnote to it.

- That the dependency graph is causally necessary for detection. On the one case where both were measured, a changed-files rule would have tied.
- Any savings in anyone's production CI. Nothing has ever been skipped in a real pipeline; every reduction here is a controlled replay in DiffCI's sandbox with both paths executed.
- A general prediction accuracy rate for the economic predictor. One out-of-sample sign match is $n = 1$.
- How often the predictor can be reached. 1 of 5 externally selected repositories was assessable; that is a signal, not a population estimate.
- A false-green rate. 32 measurable cases with 0 misses is a count, not a reliability estimate.
- That DiffCI is cheaper than a path rule in general. It lost on hono, won on zod, immer and eslint-plugin-jest, and has not been measured on enough repositories to say.
- Anything about repositories outside TypeScript and JavaScript, or without a tsconfig.json. They are refused, not measured.
- Any customer, user, pilot or revenue. As of 3 September 2026 there were zero external installs, confirmed by direct database query.

Download and cite

Published under the Creative Commons Attribution 4.0 International (CC BY 4.0). Reproduce, adapt or republish any of it, including commercially, with credit to DiffCI and a link to the study page. The licence covers the study materials only, not the DiffCI software or the named repositories' code.

CSV of all 102 figures: <https://diffci.com/research/2026/diffci-open-evidence-2026.csv>

Study page: <https://diffci.com/research/diffci-open-evidence-2026>

Licence: <https://creativecommons.org/licenses/by/4.0/>

Editorial article (PDF): <https://diffci.com/research/2026/diffci-path-rule-article.pdf>

Evidence levels used in the CSV: MEASURED, a clock or a counter produced the number in a real execution; PREDICTED, a frozen rule's prediction, committed before the measurement it predicted; PROCESS_FACT, something that happened rather than a measurement; ABSTAINED, DiffCI declined to produce a number and the abstention is the finding. Each row names the internal report that produced it; those reports are not yet public, and an evidence bundle of the frozen methodologies, aggregate inputs and results, manifests and checksums is the planned next release under this licence.

Suggested citation:

DiffCI. (2026). DiffCI Open Evidence Study 2026 (diffci-oes-2026-v1): What change-aware test selection is actually worth on real open-source CI, measured, including where it is worth nothing. <https://diffci.com/research/diffci-open-evidence-2026>